



**DIKTISAINTEK
BERDAMPAK**

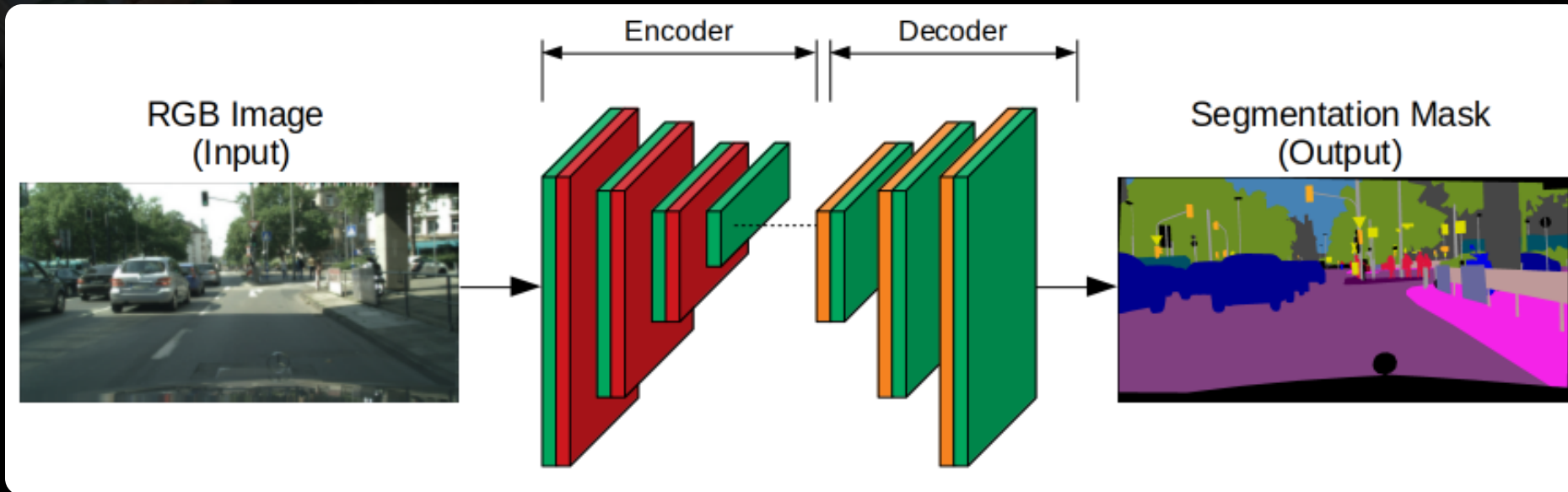
COMPUTER VISION (TFP505)

Pertemuan 4 – Basic Segmentation

ALIFIA REVAN PRANANDA

Department of Information Technology
Faculty of Engineering
Universitas Tidar

IMAGE SEGMENTATION



**WHY IS IT
IMPORTANT ?**

**HOW DOES IT
WORK ?**

In certain cases, such as finding tumors in medical images, finding targets in satellite/aerial images, finding people in surveillance images, summarizing video, we should find a specific object for better recognition.

Hence, we need to conduct image segmentation. The purpose of image segmentation is to partition an image into **meaningful regions** with respect to a particular application. The segmentation is based on measurements taken from the image and might be greylevel, color, texture, depth or motion

IMAGE SEGMENTATION

❖ Group similar components (such as, pixels in an image, image frames in a video) to obtain a compact representation.

- ❑ Segmentation is the *first step* for image analysis.
- ❑ Segmentation is used to *subdivide* an image into its constituent parts or objects.
- ❑ This step determines the eventual success or failure of image analysis.
- ❑ Generally, the segmentation is *carried out only up to the objects of interest are isolated*. e.g. face detection.
- ❑ The goal of segmentation is to *simplify and/or change the representation of an image into something that is more meaningful* and easier to analyze.

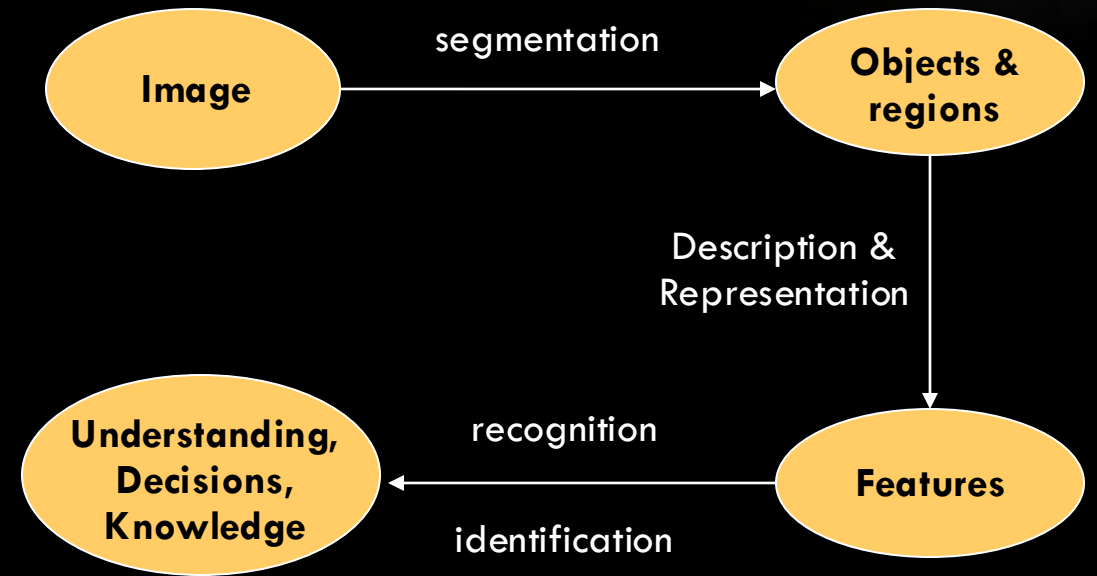
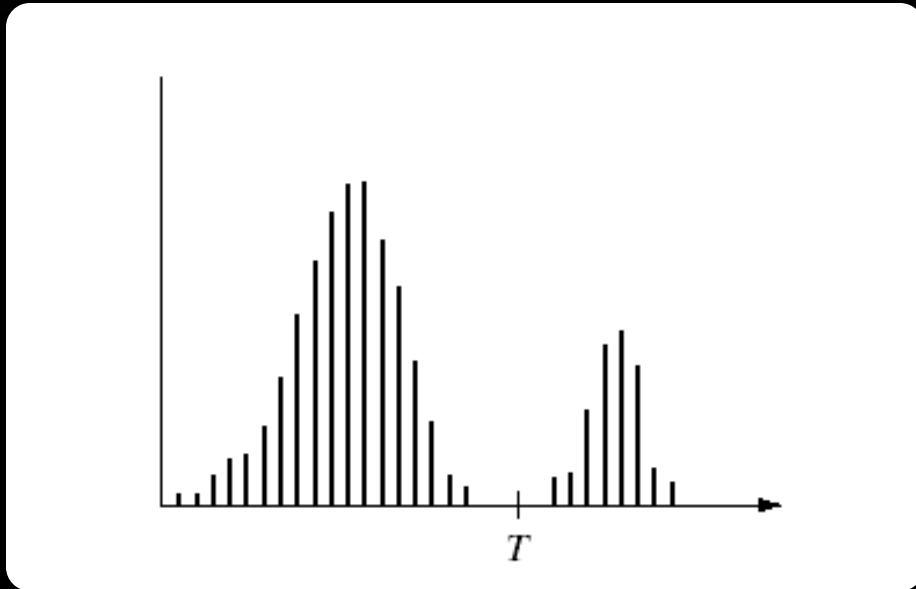


IMAGE SEGMENTATION

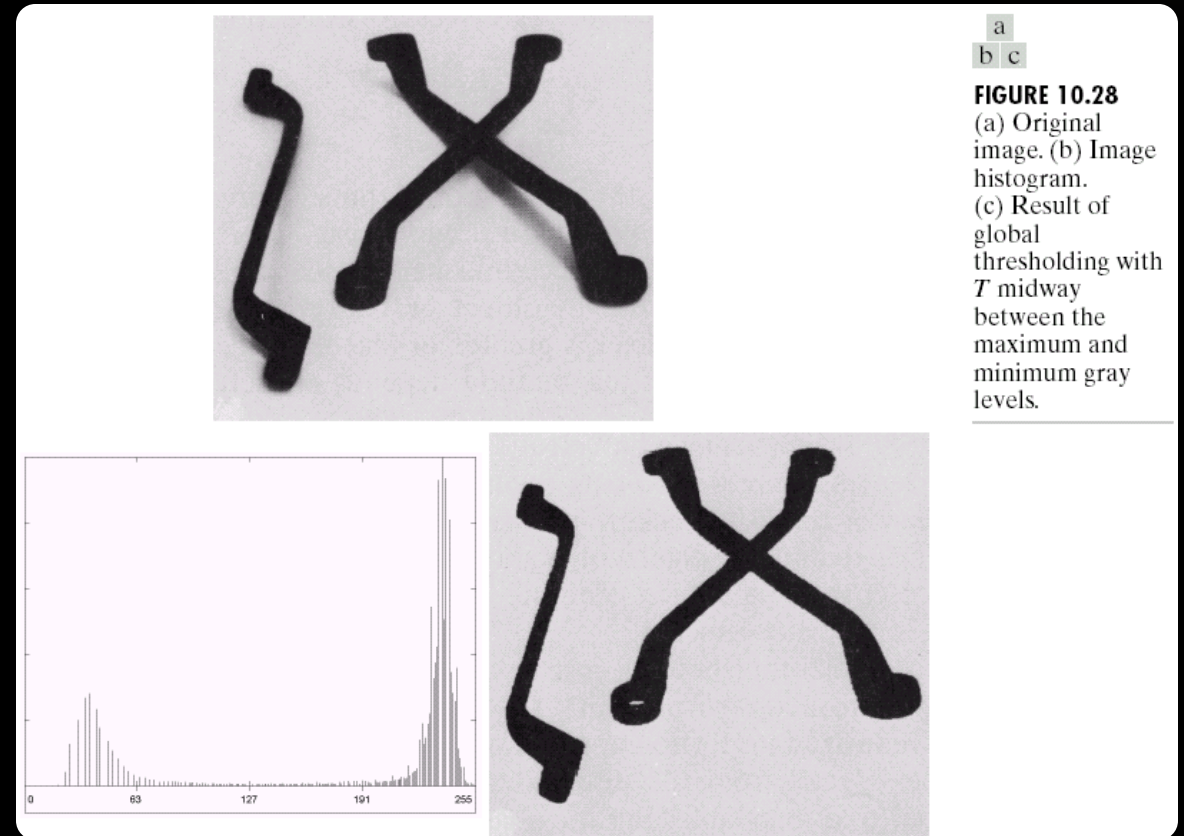
- Segmentation algorithms for monochrome images generally are based on one of two basic properties of gray-scale values:
 - **Discontinuity**
 - The approach is to partition an image based on abrupt changes in gray-scale levels.
 - The principal areas of interest within this category are detection of isolated points, lines, and edges in an image.
 - **Similarity**
 - The principal approaches in this category are based on thresholding, region growing, and region splitting/merging.

THRESHOLDING

- Suppose that an image, $f(x,y)$, is composed of light objects on a dark background, and the following figure is the histogram of the image.



- Then, the objects can be extracted by comparing pixel values with a threshold T .



THRESHOLDING

```
# Install library yang dibutuhkan
!pip install opencv-python matplotlib --quiet

import cv2
import matplotlib.pyplot as plt
from google.colab import files
import numpy as np

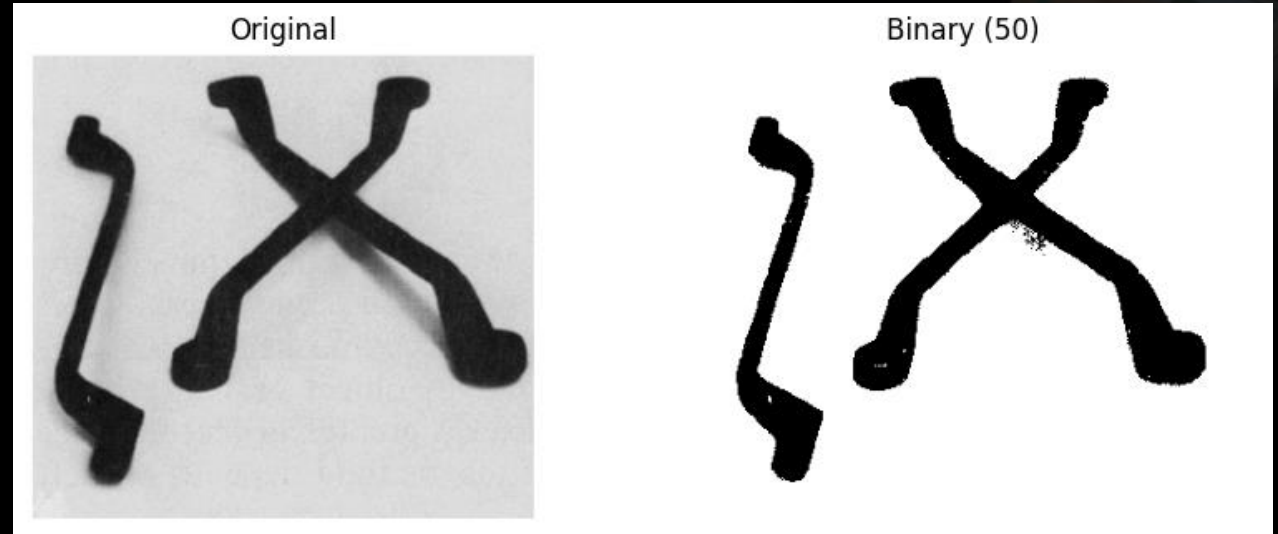
# Upload gambar dari komputer
uploaded = files.upload()

# Ambil nama file gambar yang diupload
for filename in uploaded.keys():
    img_path = filename

# Baca gambar dalam grayscale
img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

# Thresholding global
_, th_binary = cv2.threshold(img, 50, 255, cv2.THRESH_BINARY)

# Tampilkan hasil
titles = ['Original', 'Binary (50)']
images = [img, th_binary]
```



- We can set this value depend on the pixel distribution

THRESHOLDING

```
# Install library yang dibutuhkan
!pip install opencv-python matplotlib --quiet

import cv2
import matplotlib.pyplot as plt
from google.colab import files
import numpy as np

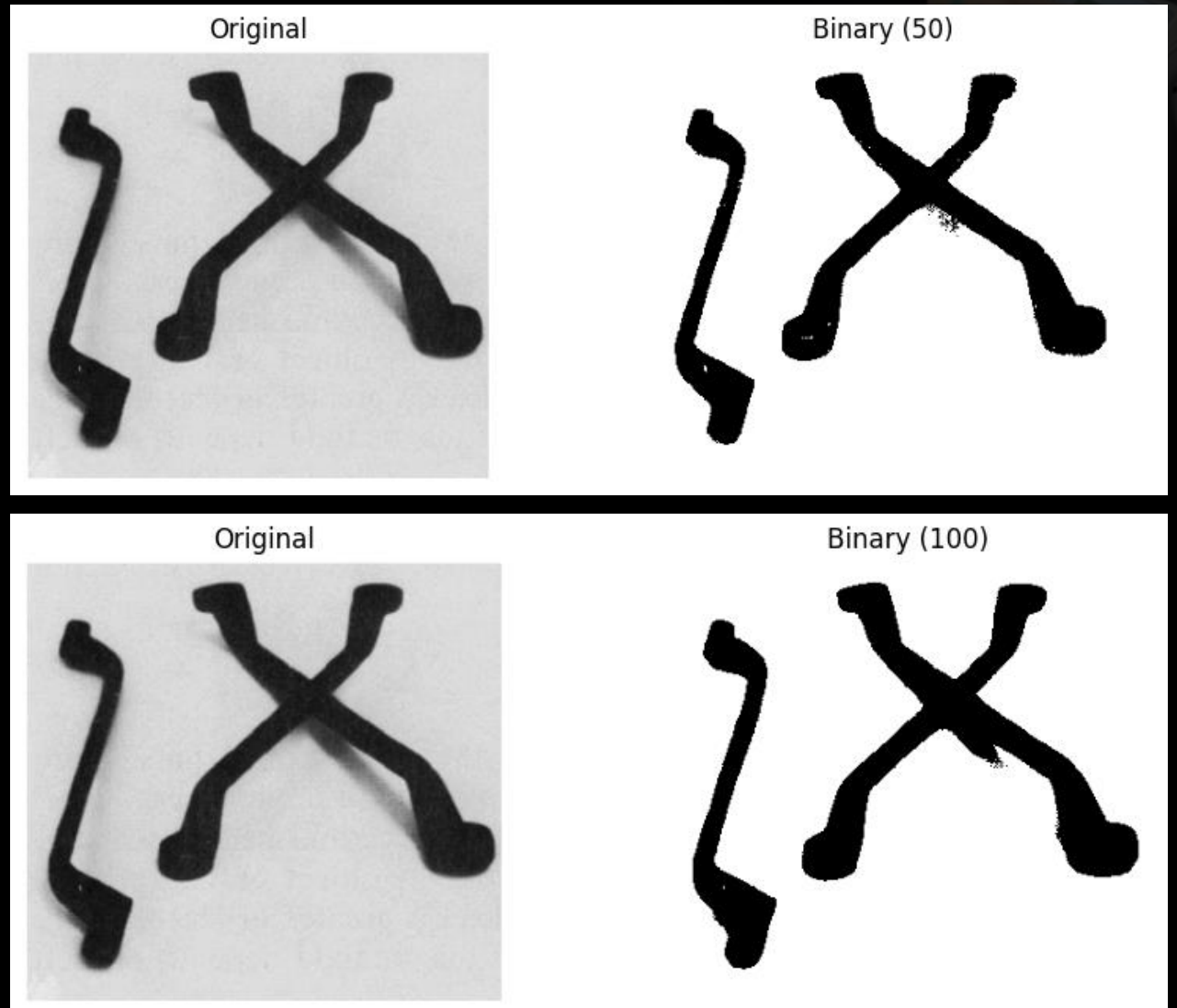
# Upload gambar dari komputer
uploaded = files.upload()

# Ambil nama file gambar yang diupload
for filename in uploaded.keys():
    img_path = filename

# Baca gambar dalam grayscale
img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

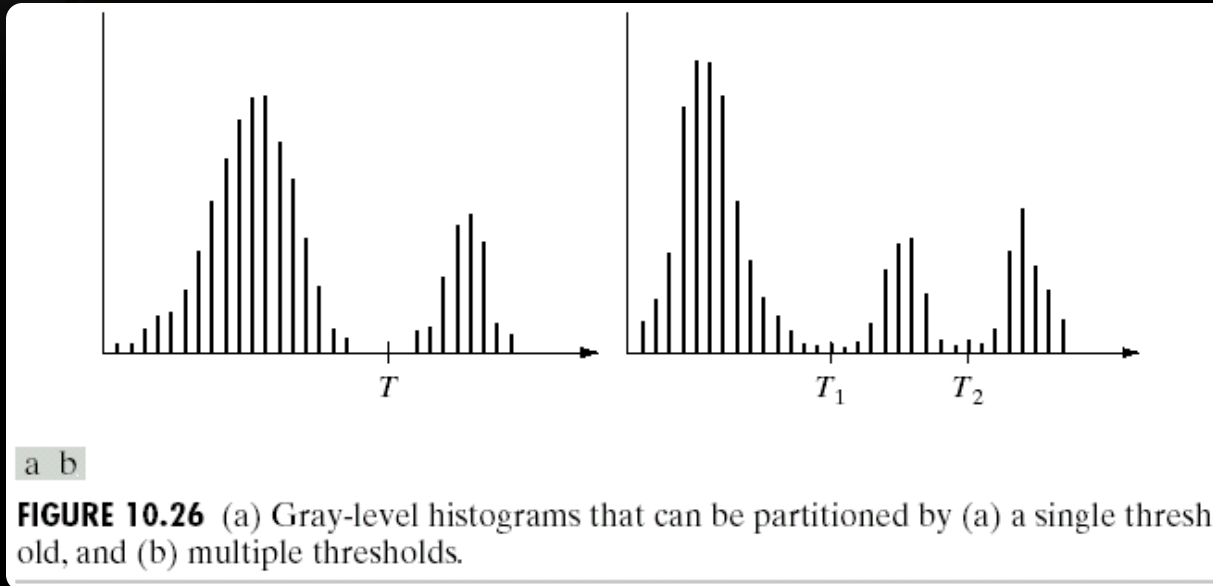
# Thresholding global
_, th_binary = cv2.threshold(img, 100, 255, cv2.THRESH_BINARY)

# Tampilkan hasil
titles = ['Original', 'Binary (50)']
images = [img, th_binary]
```



THRESHOLDING

- It is also possible to extract objects that have a specific intensity range using multiple thresholds.

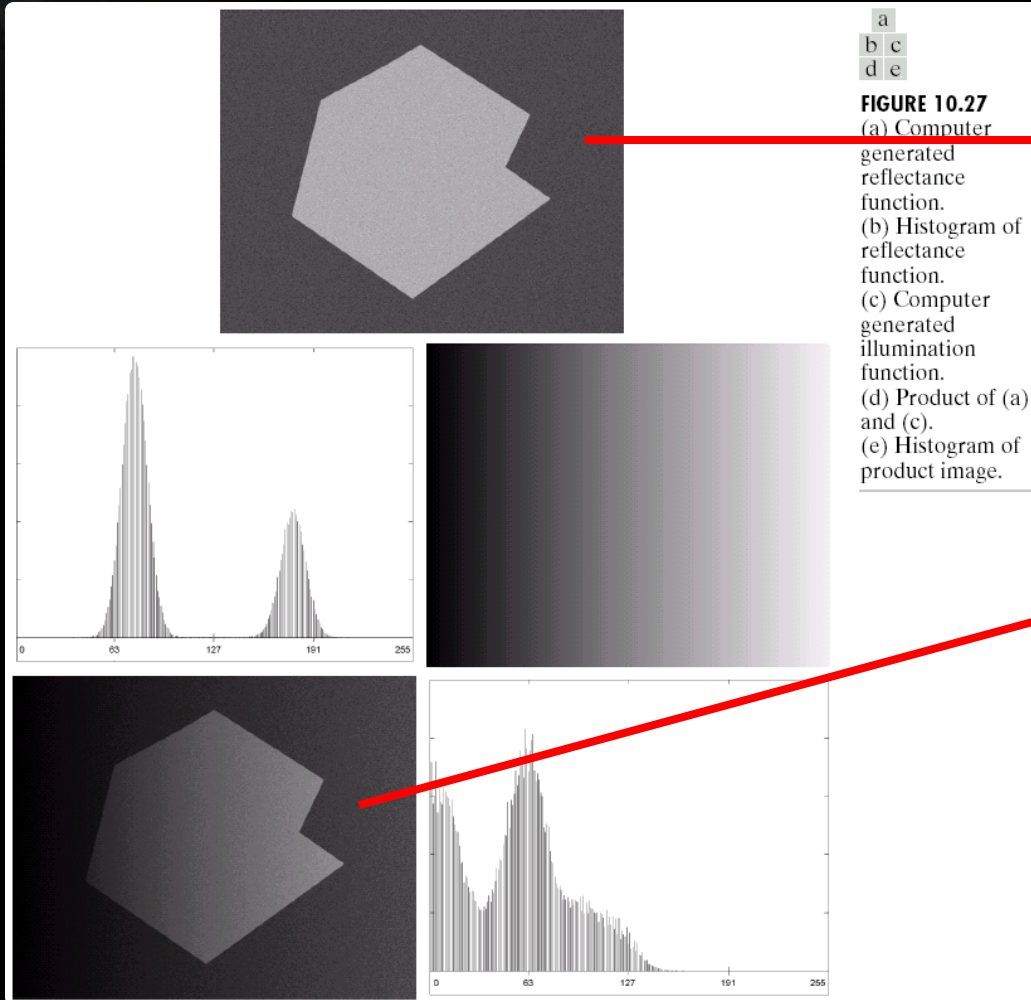


Extension to color images is straightforward:

- ❑ There are three color channels, in each one specify the intensity range of the object
 - ❑ Even if objects are not separated in a single channel, they might be with all the channels
 - ❑ Application example: Detecting/Tracking faces based on skin color
- Non-uniform illumination may change the histogram in a way that it becomes impossible to segment the image using a single global threshold.
 - Choosing local threshold values may help.

THRESHOLDING

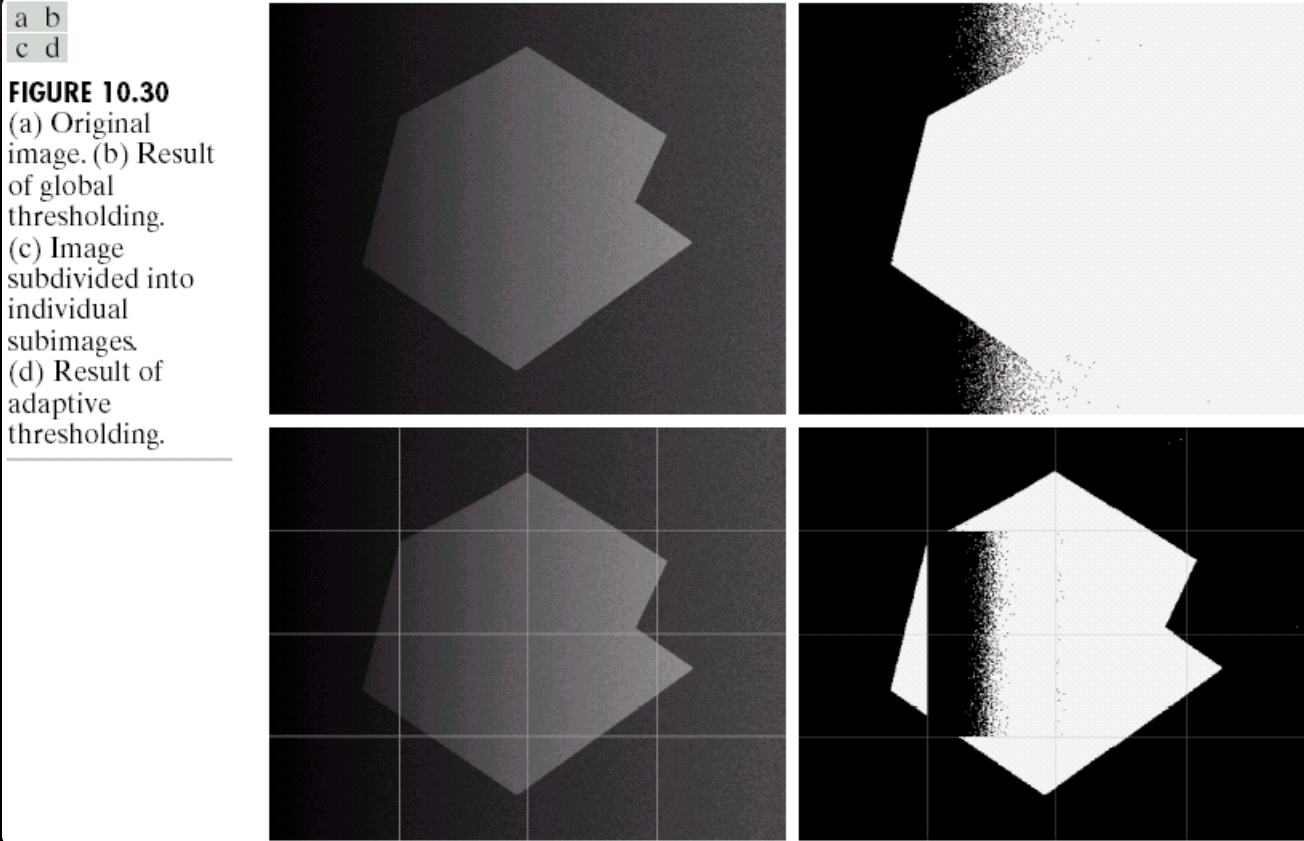
■ Example:



- For this image, we can employ single threshold value as we have practice before since it consist of two-pixel value (background and object)
- For this image, we cannot employ single threshold value since it consist of various pixel value (left background is more dark than other side)

THRESHOLDING

- Example:



- This picture illustrates the results if we apply single threshold value
- To overcome this cases, we can employ multi-thresholding method
- Multi-thresholding method enable us to apply two or more threshold value to segment the object

THRESHOLDING

```
# Install library yang dibutuhkan
!pip install opencv-python matplotlib --quiet

import cv2
import matplotlib.pyplot as plt
from google.colab import files
import numpy as np

# Upload gambar dari komputer
uploaded = files.upload()

# Ambil nama file gambar yang diupload
for filename in uploaded.keys():
    img_path = filename

# Baca gambar dalam grayscale
img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

# Threshold global dengan 2 nilai ambang
lower = 70 # batas bawah (ubah sesuai kebutuhan)
upper = 160 # batas atas (ubah sesuai kebutuhan)

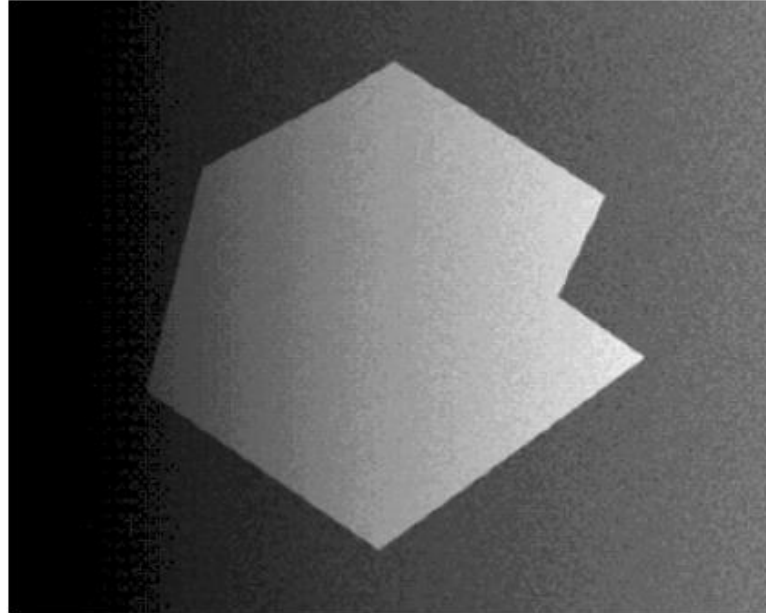
# Hasil thresholding (range)
mask = cv2.inRange(img, lower, upper)

# Tampilkan hasil
plt.figure(figsize=(12,6))
plt.subplot(1,2,1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis("off")

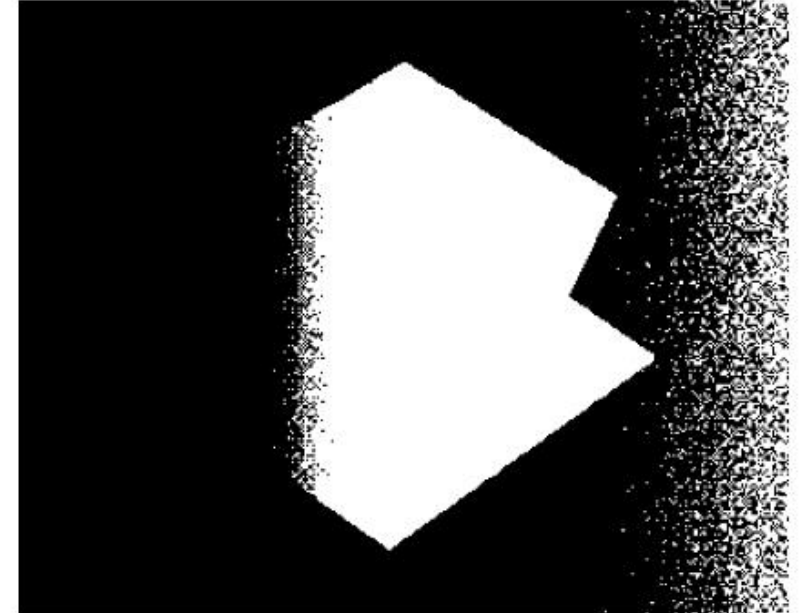
plt.subplot(1,2,2)
plt.imshow(mask, cmap='gray')
plt.title(f"Double Threshold [{lower}, {upper}]")
plt.axis("off")

plt.show()
```

Original



Double Threshold [70, 160]



This result indicates that two threshold values cannot produce smooth segmentation result.

For getting better result, we can also add the number of threshold value

THRESHOLDING

```
# Install library yang dibutuhkan
!pip install opencv-python matplotlib --quiet

import cv2
import matplotlib.pyplot as plt
from google.colab import files
import numpy as np

# Upload gambar dari komputer
uploaded = files.upload()

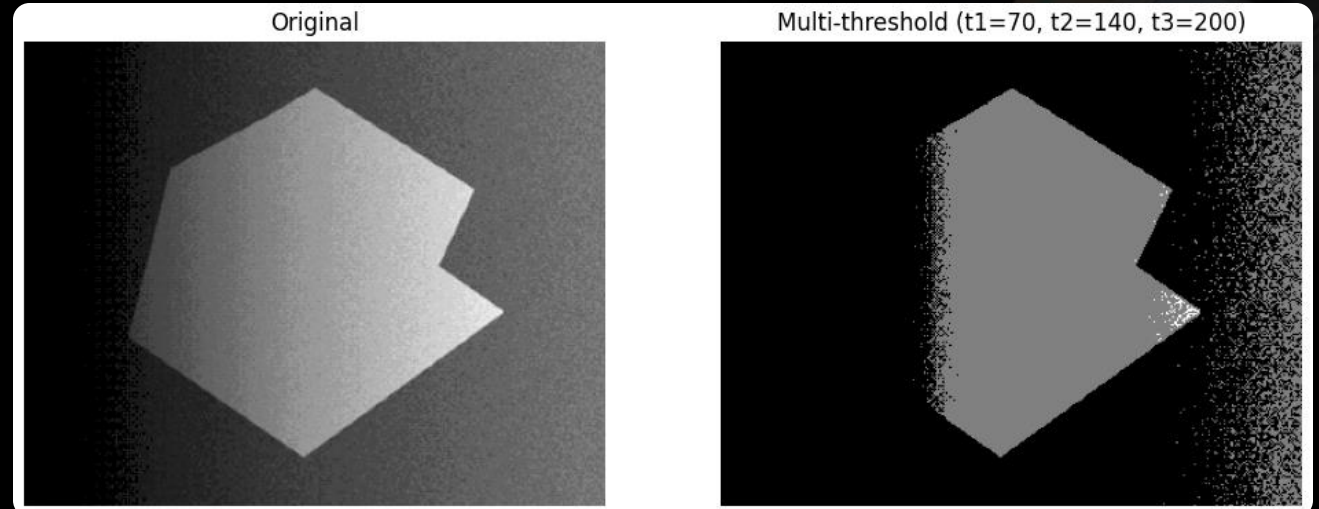
# Ambil nama file gambar yang diupload
for filename in uploaded.keys():
    img_path = filename

# Baca gambar dalam grayscale
img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

# Tiga nilai threshold
t1, t2, t3 = 70, 140, 200

# Buat citra kosong
result = np.zeros_like(img)

# Mapping sesuai range
result[(img >= 0) & (img < t1)] = 0      # hitam
result[(img >= t1) & (img < t2)] = 85    # abu-abu muda
result[(img >= t2) & (img < t3)] = 170   # abu-abu lebih terang
result[(img >= t3)] = 255                # putih
```



This picture illustrates the result of using three threshold values

THRESHOLDING

```
# Install library yang dibutuhkan
!pip install opencv-python matplotlib scikit-image --quiet

import cv2
import matplotlib.pyplot as plt
from google.colab import files
import numpy as np
from skimage.filters import threshold_multiotsu

# Upload gambar dari komputer
uploaded = files.upload()

# Ambil nama file gambar yang diupload
for filename in uploaded.keys():
    img_path = filename
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

# --- 1. Global Otsu ---
_, th_otsu = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

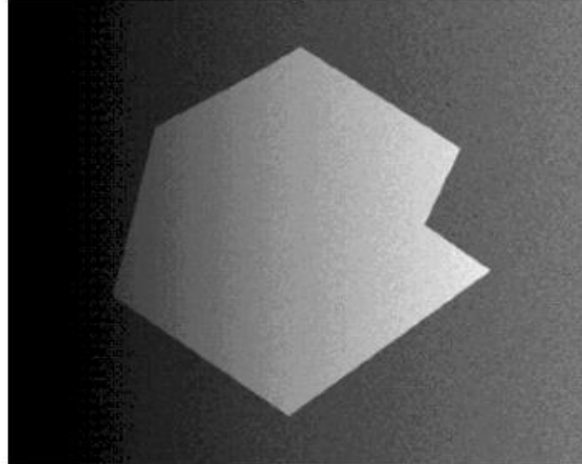
# --- 2. Multi-Otsu otomatis ---
# Tentukan jumlah kelas (contoh: 3 kelas = 2 threshold)
thresholds = threshold_multiotsu(img, classes=3)
multi_otsu = np.digitize(img, bins=thresholds) * (255 // (len(thresholds)+1))

# --- Tampilkan hasil ---
titles = ["Original", "Global Otsu", "Multi-Otsu"]
images = [img, th_otsu, multi_otsu]

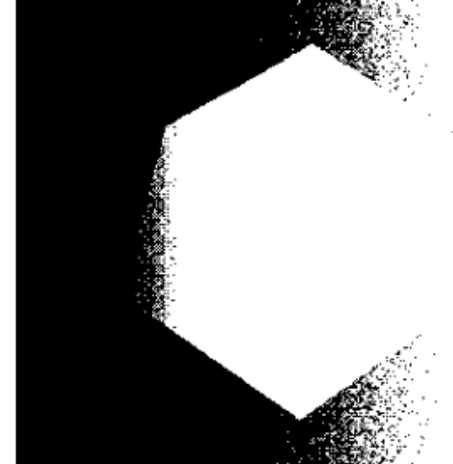
plt.figure(figsize=(15,8))
for i in range(len(images)):
    plt.subplot(2,2,i+1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")
plt.show()

print("Thresholds (Multi-Otsu):", thresholds)
```

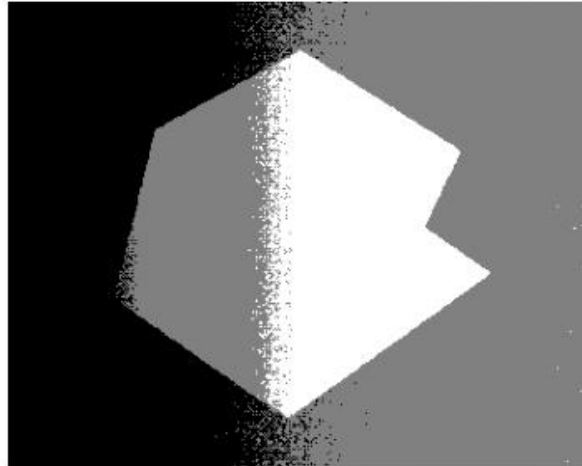
Original



Global Otsu



Multi-Otsu



Thresholds (Multi-Otsu): [36 83]

Other result using
Global Otsu and
Multi-Otsu approach

REGION-ORIENTED SEGMENTATION

■ Region Splitting

- Region growing starts from a set of seed points.
- An alternative is to start with the whole image as a single region and subdivide the regions that do not satisfy a condition of homogeneity.

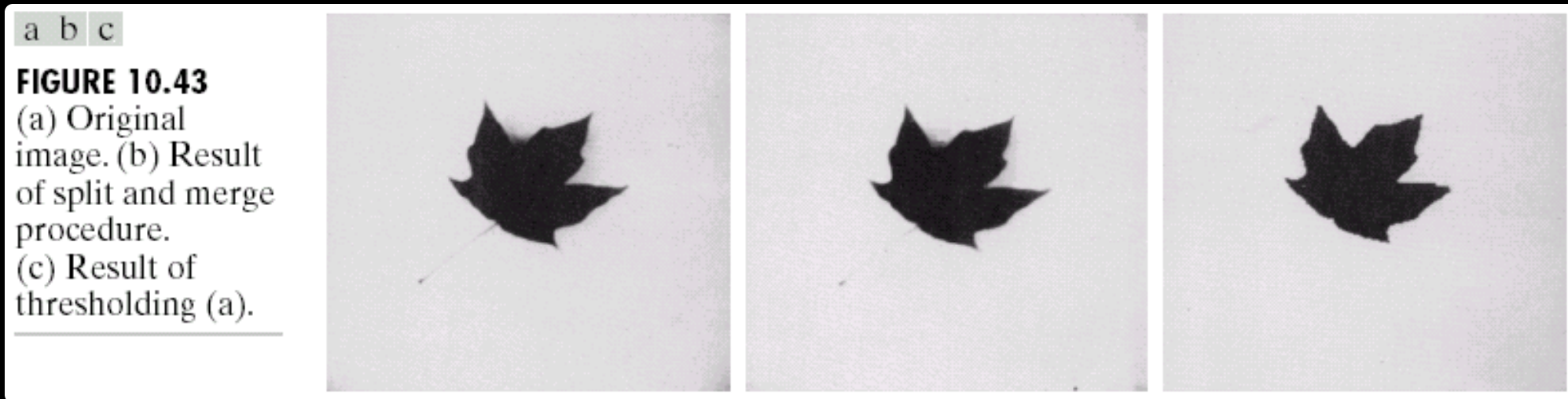
■ Region Merging

- Region merging is the opposite of region splitting.
- Start with small regions (e.g. 2x2 or 4x4 regions) and merge the regions that have similar characteristics (such as gray level, variance).
- Typically, splitting and merging approaches are used iteratively.

REGION-ORIENTED SEGMENTATION

■ Region Growing

- Region growing is a procedure that groups pixels or subregions into larger regions.
- The simplest of these approaches is *pixel aggregation*, which starts with a set of “seed” points and from these grows regions by appending to each seed points those neighboring pixels that have similar properties (such as gray level, texture, color, shape).
- Region growing based techniques are better than the edge-based techniques in noisy images where edges are difficult to detect.



REGION-ORIENTED SEGMENTATION

```
from google.colab import files
import numpy as np
from skimage.filters import threshold_multiotsu

# Upload gambar dari komputer
uploaded = files.upload()

# Ambil nama file gambar yang diupload
for filename in uploaded.keys():
    img_path = filename
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

# Seed point (bisa ubah sesuai kebutuhan)
seed = (100, 100) # (y, x)
threshold = 15 # toleransi perbedaan intensitas
```

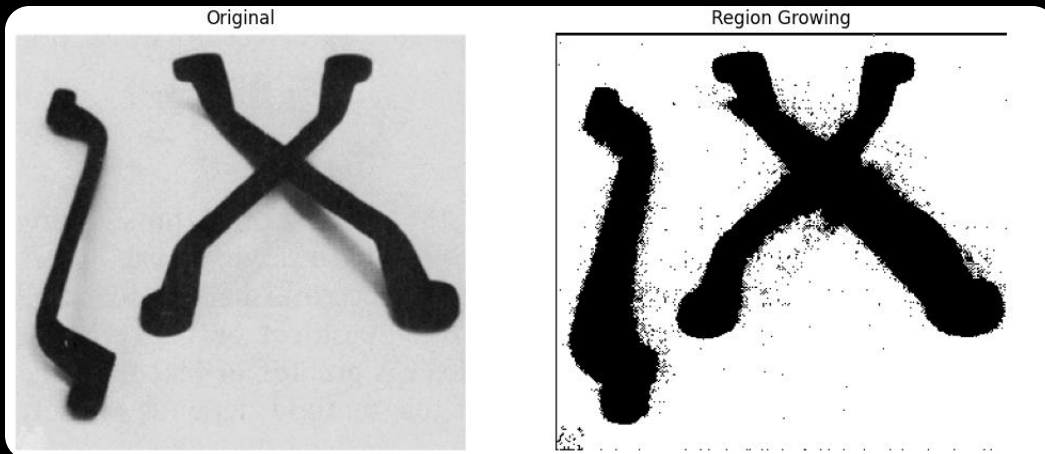
```
# Region Growing
h, w = img.shape
visited = np.zeros_like(img, dtype=bool)
segmented = np.zeros_like(img, dtype=np.uint8)

seed_value = img[seed]
queue = deque([seed])
visited[seed] = True

while queue:
    y, x = queue.popleft()
    segmented[y, x] = 255
    for dy in [-1, 0, 1]:
        for dx in [-1, 0, 1]:
            ny, nx = y + dy, x + dx
            if 0 <= ny < h and 0 <= nx < w and not visited[ny, nx]:
                if abs(int(img[ny, nx]) - int(seed_value)) < threshold:
                    visited[ny, nx] = True
                    queue.append((ny, nx))
```

```
# Tampilkan hasil
plt.figure(figsize=(12,6))
plt.subplot(1,2,1)
plt.imshow(img, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(segmented, cmap="gray")
plt.title("Region Growing")
plt.axis("off")
plt.show()
```



This method starts from a set of seed points and expands the region by including neighboring pixels whose intensity values are similar to the seed. It is effective when regions have clear homogeneity and well-defined seed points.

REGION-ORIENTED SEGMENTATION

■ Region Splitting and Merging

- is a region-based image segmentation technique that recursively partitions an image into smaller regions and then combines them based on homogeneity criteria. The main idea is *to ensure that each segmented region is uniform in terms of intensity, color, or texture*.
- **The process typically follows two steps:**
 - **Splitting:** The image is divided into four quadrants (quadtree decomposition). If a region is not homogeneous—meaning the variance of pixel values within the region is higher than a defined threshold—it is further split into four smaller sub-regions. This recursive process continues until all regions are either homogeneous or reach a predefined minimum size.
 - **Merging:** After splitting, adjacent regions that are similar enough (for example, having a small variance difference or similar mean intensity) are merged together. This step reduces over-segmentation caused by excessive splitting and ensures larger homogeneous regions.

Region Splitting and Merging is particularly useful for images with non-uniform areas where global thresholding fails to produce meaningful results. Unlike edge-based segmentation, which focuses on detecting boundaries, this method emphasizes the homogeneity of pixel groups, making it more robust in the presence of noise or gradual intensity variations

REGION-ORIENTED SEGMENTATION

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from collections import deque
from google.colab import files
import numpy as np
from skimage.filters import threshold_multiotsu

# Upload gambar dari komputer
uploaded = files.upload()

# Ambil nama file gambar yang diupload
for filename in uploaded.keys():
    img_path = filename
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

# Threshold variansi untuk cek homogenitas
var_threshold = 500.0

# Recursive splitting function
def split_region(img, x, y, w, h, min_size=16):
    region = img[y:y+h, x:x+w]
    if region.size == 0:
        return []

    # cek homogenitas (variansi)
    if np.var(region) < var_threshold or w <= min_size or h <= min_size:
        return [(x, y, w, h)]

    # bagi jadi 4 kuadran
    half_w, half_h = w // 2, h // 2
    regions = []
    regions += split_region(img, x, y, half_w, half_h, min_size)
    regions += split_region(img, x + half_w, y, half_w, half_h, min_size)
    regions += split_region(img, x, y + half_h, half_w, half_h, min_size)
    regions += split_region(img, x + half_w, y + half_h, half_w, half_h, min_size)

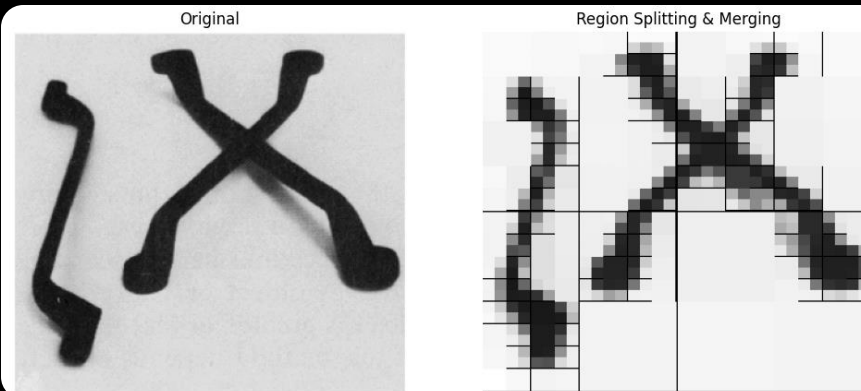
    return regions
```

```
# Jalankan splitting
h, w = img.shape
regions = split_region(img, 0, 0, w, h)

# Buat citra hasil segmentasi
segmented = np.zeros_like(img)
for (x, y, rw, rh) in regions:
    mean_val = int(np.mean(img[y:y+rh, x:x+rw]))
    segmented[y:y+rh, x:x+rw] = mean_val

# Tampilkan hasil
plt.figure(figsize=(12,6))
plt.subplot(1,2,1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(segmented, cmap='gray')
plt.title("Region Splitting & Merging")
plt.axis("off")
plt.show()
```

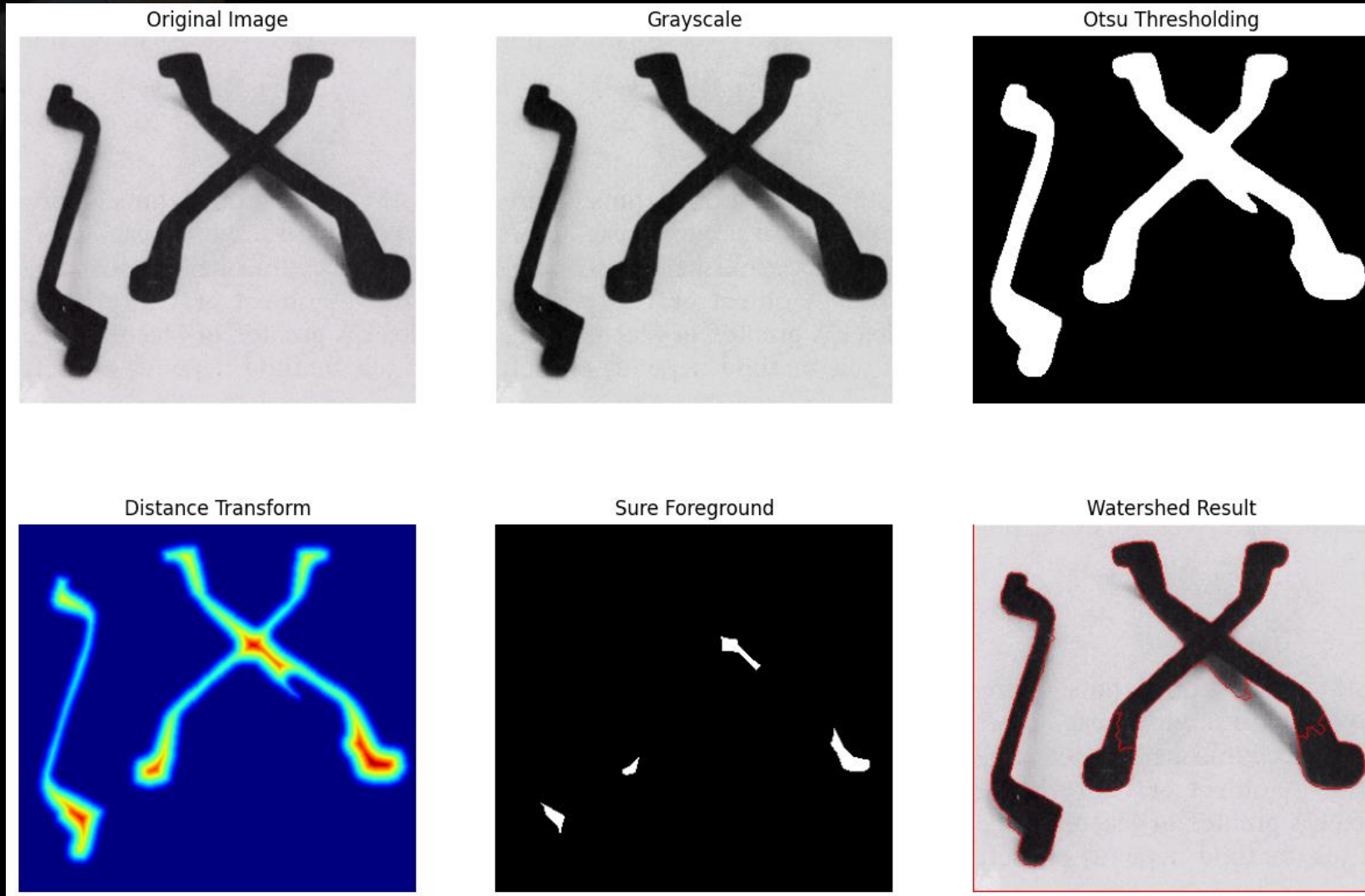


REGION-ORIENTED SEGMENTATION

Watershed Segmentation is a region-based image segmentation technique inspired by topographic interpretation of grayscale images. In this approach, the image is considered as a three-dimensional landscape, where the intensity of each pixel represents elevation: high intensity values correspond to peaks (ridges) and low intensity values correspond to valleys (basins).

The standard watershed algorithm can lead to **over-segmentation**, especially in noisy images or images with many small variations in intensity. To overcome this, preprocessing steps such as noise removal, morphological filtering, and marker-based watershed are often applied. In the **marker-based approach**, foreground objects and background areas are identified beforehand using morphological operations, distance transforms, or connected component analysis. These markers guide the algorithm, reducing the risk of excessive or meaningless segmentation.

REGION-ORIENTED SEGMENTATION



REGION-ORIENTED SEGMENTATION

Active Contour Models (Snakes) are a region-based image segmentation method that uses deformable curves to detect object boundaries in an image. The main idea is to start with an initial contour, such as a circle or rectangle around the object of interest and iteratively evolve this contour until it aligns with the object's edges or regions.

The contour evolution is guided by minimizing an energy function, which consists of:

- Internal energy – controls the smoothness and continuity of the curve by penalizing sharp bends or irregularities.
- External energy – attracts the contour toward important image features, such as edges (high gradients) or homogeneous regions.

There are two major types of active contour models:

- Edge-based snakes, which rely on image gradients to attract the contour to object boundaries. These are effective when the edges in the image are clear and well-defined.
- Region-based snakes (e.g., the Chan-Vese model), which do not depend on gradient information. Instead, they divide the image into regions of different intensity or texture. This makes them more robust when edges are weak, blurred, or noisy.

Active contour models are widely used in medical imaging, object detection, and shape analysis because they can accurately adapt to complex and irregular object boundaries.

REGION-ORIENTED SEGMENTATION

```
!pip install scikit-image matplotlib --quiet
import matplotlib.pyplot as plt
from skimage import io, color, img_as_float
from skimage.segmentation import chan_vese
from google.colab import files

# Upload image (or use sample if none provided)
print("Upload your image (or cancel to use skimage sample):")
uploaded = files.upload()
if uploaded:
    img_path = list(uploaded.keys())[0]
    image = io.imread(img_path)
else:
    from skimage.data import camera
    image = camera()

# Convert to grayscale & float
if image.ndim == 3:
    image = color.rgb2gray(image)
image = img_as_float(image)
```

```
# Chan-Vese segmentation
cv_result = chan_vese(image, mu=0.25, lambda1=1, lambda2=1, tol=1e-3,
                      max_num_iter=200, dt=0.5, init_level_set="checkerboard")

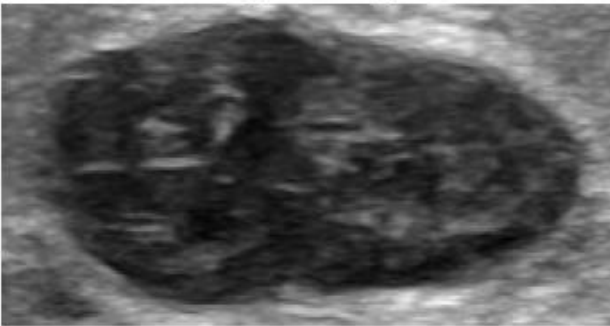
# Plot result
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(image, cmap="gray")
axes[0].set_title("Original Image")
axes[0].axis("off")

axes[1].imshow(cv_result, cmap="gray")
axes[1].set_title("Chan-Vese Segmentation")
axes[1].axis("off")

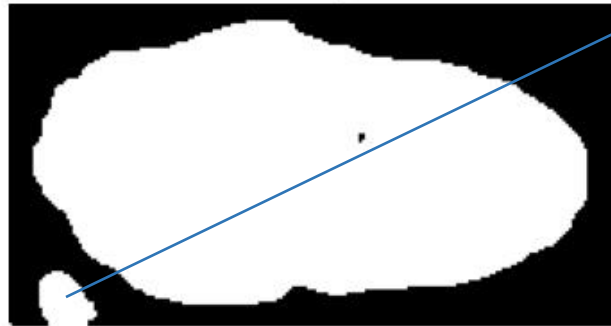
axes[2].imshow(image, cmap="gray")
axes[2].contour(cv_result, [0.5], colors='r')
axes[2].set_title("Segmentation Boundary")
axes[2].axis("off")

plt.show()
```

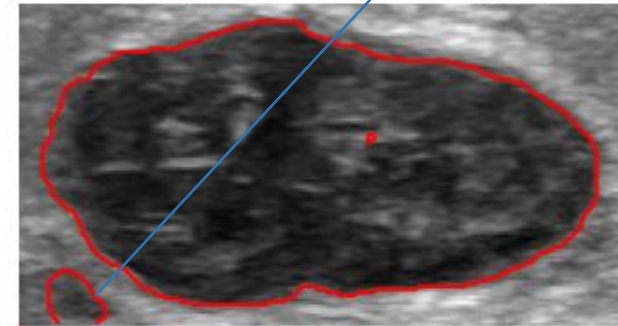
Original Image



Chan-Vese Segmentation



Segmentation Boundary



This original method remain an unwanted object

We can modify the result by removing the small object

REGION-ORIENTED SEGMENTATION

```
!pip install scikit-image matplotlib --quiet

import matplotlib.pyplot as plt
from skimage import io, color, img_as_float
from skimage.segmentation import chan_vese
from skimage.measure import label, regionprops
from skimage.morphology import closing, disk, remove_small_objects
from google.colab import files

# Upload image (or use sample if none provided)
print("Upload your image (or cancel to use skimage sample):")
uploaded = files.upload()
if uploaded:
    img_path = list(uploaded.keys())[0]
    image = io.imread(img_path)
else:
    from skimage.data import camera
    image = camera()

# --- Load image ---
if image.ndim == 3:
    image = color.rgb2gray(image)
image = img_as_float(image)

# --- Chan-Vese segmentation ---
cv_result = chan_vese(image, mu=0.5, lambda1=1, lambda2=1, tol=1e-3,
                      max_num_iter=500, dt=0.5, init_level_set="checkerboard")
```

```
# --- Remove small objects (threshold 500 pixels, bisa diubah sesuai kebutuhan)
cleaned = remove_small_objects(cv_result.astype(bool), min_size=500)

# --- Keep only largest object ---
labels = label(cleaned)
regions = regionprops(labels)
if regions:
    largest_region = max(regions, key=lambda r: r.area)
    largest_mask = labels == largest_region.label
else:
    largest_mask = cleaned

# --- Optional: smooth boundary with morphological closing ---
largest_mask = closing(largest_mask, disk(5))

# --- Plot results ---
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(image, cmap="gray")
axes[0].set_title("Original Image")
axes[0].axis("off")

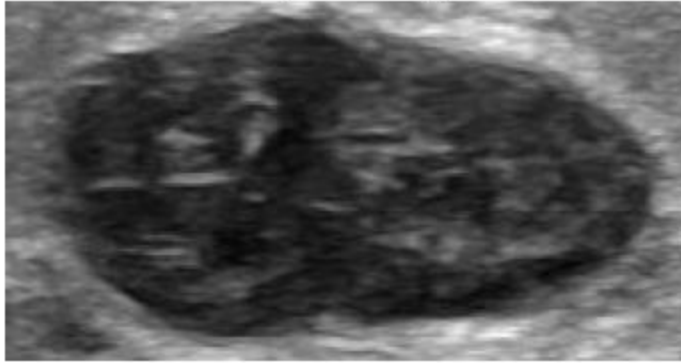
axes[1].imshow(cleaned, cmap="gray")
axes[1].set_title("After Removing Small Objects")
axes[1].axis("off")

axes[2].imshow(image, cmap="gray")
axes[2].contour(largest_mask, [0.5], colors='r')
axes[2].set_title("Final Segmentation Boundary")
axes[2].axis("off")

plt.show()
```

REGION-ORIENTED SEGMENTATION

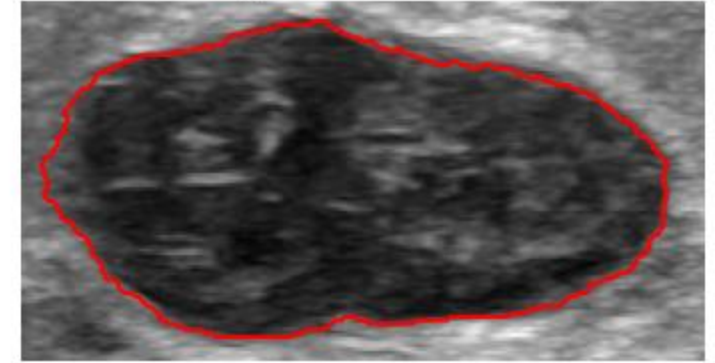
Original Image



After Removing Small Objects



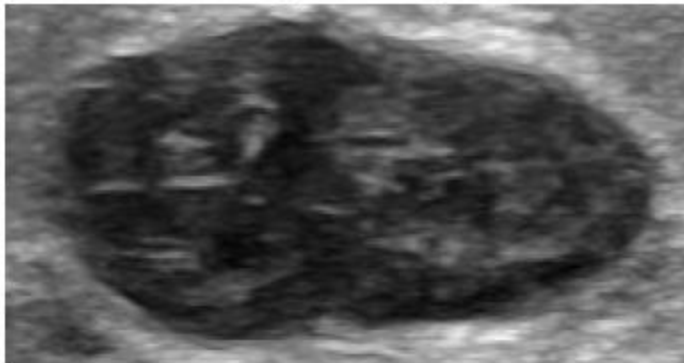
Final Segmentation Boundary



```
# --- Fill holes ---  
filled = binary_fill_holes(cleaned) # alternatif: remove_small_holes(cleaned, area_threshold=1000)
```

We can add filling method for handling the holes in segmented object

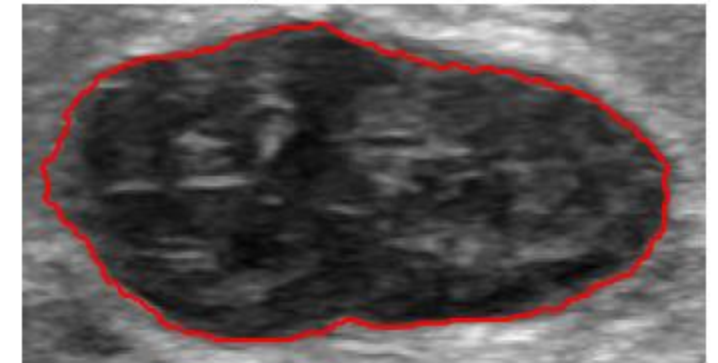
Original Image



After Filling Holes



Final Segmentation Boundary



TUGAS

Lakukan segmentasi pada gambar berikut. Lakukan preprocessing seperti image filtering apabila dibutuhkan

